

Machine Learning Technologies

Principal Component Analysis

Donny Hurley

ATU

Higher Dimensions (features)

The breast cancer data set had 30 features. Those are manageable.

But it's easy to end up with lots more input features than that, particularly with images.

Another built-in data source: images of famous people, labelled with their names from the Labelled Faces in the Wild project.

```
from sklearn.datasets import fetch_lfw_people
faces = fetch_lfw_people(min_faces_per_person=60, resize=1.0)
print(faces.target_names)
print(faces.images.shape)

['Ariel Sharon' 'Colin Powell' 'Donald Rumsfeld' 'George W. Bush'
 'Gerhard Schroeder' 'Hugo Chavez' 'Junichiro Koizumi' 'Tony Blair']
(1348, 125, 94)
```

Images

1348 images, each $125 \times 94 = 11750$ pixels (greyscale).



The goal is to train a model to take the image and predict the name. Suddenly we have 11750 input features

An SVM classifier does better than I would have expected.

```
X_train, X_test, y_train, y_val = \
    train_test_split(faces.data, faces.target)

model = SVC(kernel='linear', C=2.0)
model.fit(X_train, y_train)
print(model.score(X_val, y_val))

0.833827893175
```

But we start to see a drawback of SVM: it takes ≈ 27 seconds to train the model and ≈ 8 seconds to test the score.

GaussianNB takes 258ms to train and 640ms to test.

kNearestNeighbours (k=5) takes 783ms to train and ≈ 17.5 seconds to test.

Running time depends on the number of features, so as the number of features increases, these gaps are going to get even worse.

Imagine trying to do a cross-validation on this.

Note: GaussianNB and kNearestNeighbours perform much worse, with scores of 0.439 and 0.558 on my last runs.

- What we need to do is somehow decrease the number of input features.
- They're probably not all important: is pixel $(0, 0)$ really important to the prediction? Aren't some pixels very highly correlated?
- Maybe we can do something about that.

Scaling the image

We could just scale the images to 50%. So $62 \times 47 = 2,914$ features instead.

This results in a faster time without affecting the accuracy too much

6.45 seconds to train and 1.9 seconds to evaluate for SVM. Decreases for kNN and NB as well. (It actually improved accuracy of NB quite a bit)

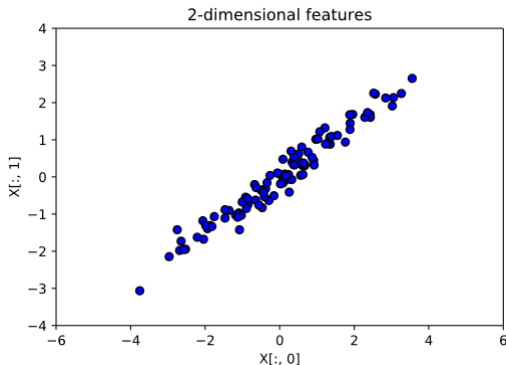
We are doing this scaling because we know the data set. Understanding your data can work better than all the algorithms but we want another way.

Something that will generalise to not just images

Principal Component Analysis (PCA) is an unsupervised learning algorithm (our first!).

The goal of PCA is to introduce a lower-dimensional representation of the dataset but still have all the information that matters.

Consider this two-dimensional data:



How two-dimensional is it really? Most of what's happening is along the diagonal. It's almost one-dimensional (but it's not the existing x or y).

Important: PCA Does NOT Use Labels

PCA ignores the target variable.

It does not know:

- What the classes are
- What we are trying to predict
- What features are useful for classification

It only maximises variance.

High variance does not always mean high predictive power.

PCA may:

- Help a classifier
- Do nothing
- Occasionally hurt performance

Principal component analysis finds the directions where most of the interesting stuff is happening (the principal components!)

The idea behind the calculation...

- Find the vector along which the data has the maximum variance: that's the first principal component.
- We find the next orthogonal direction of maximum remaining variance
- Continue until we have m components.

PCA = Rotation + Projection

PCA does two things:

- 1 Rotates the coordinate system.
- 2 Keeps only the first k new axes.

If we keep all components:

No information is lost (up to rounding error)

If we keep only a few:

We approximate the data in a lower-dimensional subspace.

So PCA does not delete features.

It rotates the space and then truncates it.

For now, lets leave it as sklearn's problem

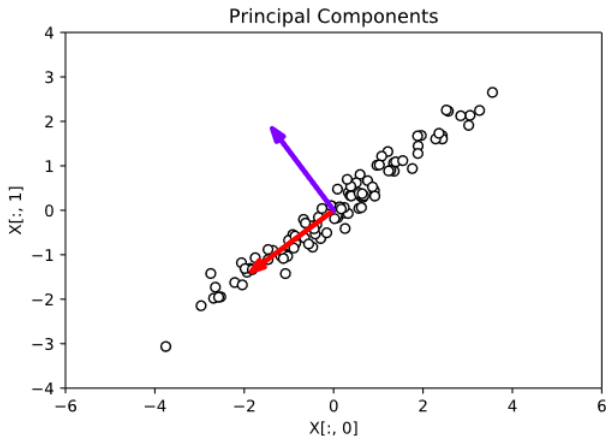
```
from sklearn.decomposition import PCA
pca = PCA(2)
pca.fit(X)
print(pca.components_)
print(pca.explained_variance_ratio_)
[[-0.80111387 -0.59851197]
 [-0.59851197  0.80111387]]
[ 0.99027873  0.00972127]
```

Output

- `pca.components_`: Principal axes in feature space, representing the directions of maximum variance in the data. The components are sorted by `explained_variance_`.
- `pca.explained_variance_ratio_`: the fraction of the variance along each axes.

99% of the variance is in the one direction

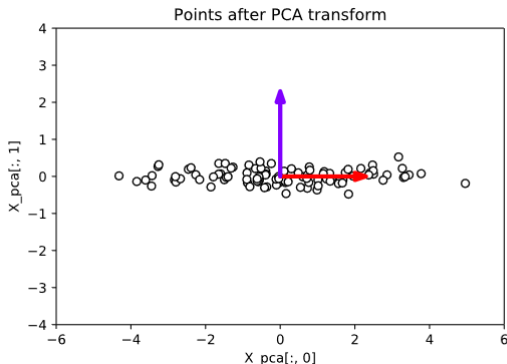
Graphically:



99% of the variance in the red direction, 1% purple

The PCA transformer can make those two things the new axes of the data.

```
X_pca = pca.transform(X)
```



If we now just use the information on the red arrow (so 1 dimension), we should be able to do a classification without losing much information.

Assuming we start m with dimensional data, and look for m principal components, we're not really losing anything (except a little floating point error).

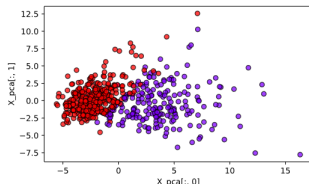
```
X_after_pca = pca.inverse_transform(pca.transform(X))  
print(np.abs(X_after_pca - X).max())
```

```
8.881784197e-16
```

The point is, we want to decrease the number of dimensions.

Sometimes, PCA can help you get a better idea what's going on with your data. On the breast cancer data:

```
scale = StandardScaler()  
pca = PCA(2)  
X_pca = pca.fit_transform(scale.fit_transform(X))
```



Obviously, some of the data in the original 30 dimensions is being lost, but we can kind of see how a model can separate the two classes. (There could be a line separating red and purple)

PCA is NOT Feature Selection

Important clarification:

- **Feature selection** = choose a subset of the original features.
- **PCA** = creates *new* features.

Each principal component is a linear combination of all original features.

For images:

$$\text{New feature}_1 = 0.02 \cdot \text{pixel}_1 - 0.01 \cdot \text{pixel}_2 + \dots$$

So PCA is better described as:

Feature Extraction

We are rotating the coordinate system, not deleting columns.

Dimensionality reduction

Probably the most useful PCA application for us: dimensionality reduction.

The face images had 11750 dimensional features. Getting rid of many of them doesn't lose much.

```
model = make_pipeline(  
    PCA(250),  
    SVC(kernel='linear', C=2.0)  
)  
model.fit(X_train, y_train)  
print(model.score(X_val, y_val))  
  
0.824925816024
```

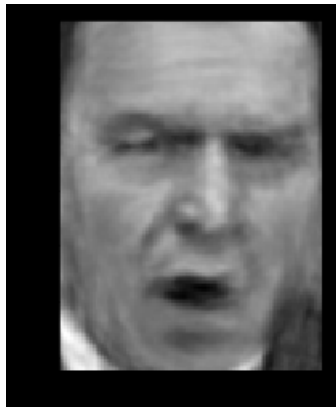
We can add up all the `explain_variance_ratio_` to see how much variance is explained by the 250 terms

With that dataset 96% of the variance is explained by the highest ranking 250 terms. So not losing that much really! But gained quite a bit of speed. On the last run

- With all features: 12.2 seconds to fit and 3.84 seconds to score
- With first 250 features: 903 ms to fit and 109 ms to score

Dimensionality reduction

What does this actually look like on pictures themselves? (The pictures were taken at random, X_{train} split means probably get different pictures if you run the workbook yourself)



Dimensionality reduction

What does this actually look like on pictures themselves? (The pictures were taken at random, X_{train} split means probably get different pictures if you run the workbook yourself)



Let's look at the `pca.components_` themselves

```
pca.components_.shape
```

```
(250, 11750)
```

So we have 250 rows, and 11750 columns.

250 principal components, but each component has 11750 features. What does that remind you of?

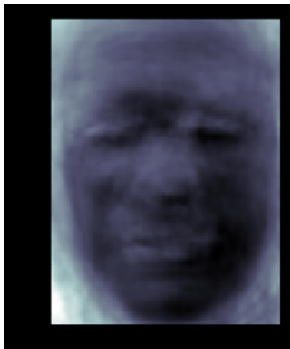
It is the same as the original number of pixels. So a `pca.component` is a "face" itself

Eigenfaces

These are often called *eigenfaces*. Mathematically, these directions turn out to be eigenvectors of a matrix built from the data.

We won't go into that machinery — but it guarantees these directions are orthogonal and optimal.

Eigenfaces are quite weird looking. Here is the "image" for the first principal component.



Eigenfaces

We can look at the first 24 eigenfaces (the code to do this is in the workbook). I'll pause before showing the horror (although could it really be worse than an image of George)



This should give us insight into how the images vary.

- The first few eigenfaces (from the top left) seem to be associated with the angle of lighting on the face.
- The later ones seem to be picking out certain features, such as eyes, noses and lips.

PCA can also be used as a filtering approach for noisy data. The idea is this:

- Any components with variance much larger than the effect of the noise should be relatively unaffected by the noise.
- So if you reconstruct the data using just the largest subset of principal components, you should be preferentially keeping the signal and throwing out the noise.

Why Does PCA Remove Noise?

Key intuition:

- Signal tends to be structured.
- Noise tends to be small and spread out in many directions.

PCA keeps directions with **large variance**.

Low-variance directions often correspond to:

- Random pixel fluctuations
- Measurement noise
- Small irregularities

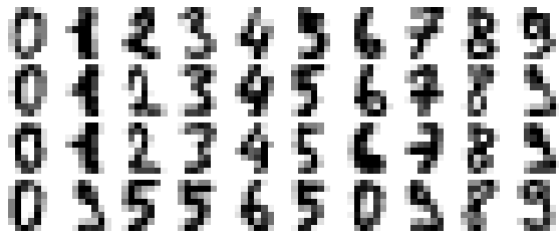
By reconstructing using only the top components:

We keep structure and discard small random variation.

That is why the digits look cleaner.

Back to digits!

Here are a sample of digits information plotted



And now adding random noise to create a noisy dataset, re-plotting gives





It's clear by eye that the images are noisy, and contain spurious pixels. Let's train a PCA on the noisy data, requesting that the projection preserve 50% of the variance:

```
pca = PCA(0.50).fit(noisy)
pca.n_components_
```

```
12
```

Here 50% of the variance amounts to 12 principal components.

Noise Filtering

Now we compute these components, and then use the inverse of the transform to reconstruct the filtered digits:

```
components = pca.transform(noisy)
filtered = pca.inverse_transform(components)
plot_digits(filtered)
```



Looks a lot better

- This signal preserving/noise filtering property makes PCA a very useful feature extraction routine
- For example, rather than training a classifier on very high-dimensional data, you might instead train the classifier on the lower-dimensional representation
- This will automatically serve to filter out random noise in the inputs.

Behind the Scenes: Singular Value Decomposition (SVD)

In practice, sklearn computes PCA using:

$$X = U\Sigma V^T$$

This is called the **Singular Value Decomposition**.

- Columns of V = principal directions
- Singular values in Σ measure importance

SVD is used in many areas of Machine Learning:

- Recommender systems (e.g. Netflix-style matrix factorisation)
- Latent Semantic Analysis in NLP
- Image compression
- Collaborative filtering

PCA is one important application of SVD.